```
!pip install gradio reportlab autopep8 black
```

```
import ast
import gradio as gr
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
import datetime
import tempfile
```

```
class DocGenieAnalyzer:

    @staticmethod
    def extract_function_signature(code):
        tree = ast.parse(code)

        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
                func_name = node.name

                params = []
                for arg in node.args.args:
                    param_name = arg.arg
                    param_type = "Any"

                    if arg.annotation:
                        param_type = ast.unparse(arg.annotation)

                    params.append({
                        "name": param_name,
                        "type": param_type,
                        "default": None
                    })

                return_type = "Any"
                if node.returns:
                    return_type = ast.unparse(node.returns)

                return {
                    "name": func_name,
                    "params": params,
                    "return_type": return_type
                }

        return None


    @staticmethod
    def analyze_function_logic(code):
        tree = ast.parse(code)

        has_loops = False
        has_conditions = False
        operations = []

        for node in ast.walk(tree):

            if isinstance(node, (ast.For, ast.While)):
                has_loops = True

            if isinstance(node, ast.If):
                has_conditions = True

            if isinstance(node, ast.BinOp):
                if isinstance(node.op, ast.Add):
                    operations.append("addition")
                if isinstance(node.op, ast.Sub):
                    operations.append("subtraction")
                if isinstance(node.op, ast.Mult):
                    operations.append("multiplication")
                if isinstance(node.op, ast.Div):
                    operations.append("division")

        description = "Performs computation"
```

```
        if has_loops:
            description += " using loops"

        if has_conditions:
            description += " with conditional logic"

        if operations:
            description += " and mathematical operations"

        return {
            "has_loops": has_loops,
            "has_conditions": has_conditions,
            "operations": operations,
            "description": description
        }


    @staticmethod
    def generate_google_docstring(signature, analysis):

        doc = '"""\n'
        doc += f"{analysis['description']}.\n\n"

        doc += "Args:\n"
        for p in signature["params"]:
            doc += f"    {p['name']} ({p['type']}): Parameter description.\n"

        doc += "\nReturns:\n"
        doc += f"    {signature['return_type']}: Result of the function.\n"

        doc += '"""'

        return doc


    @staticmethod
    def generate_numpy_docstring(signature, analysis):

        doc = '"""\n'
        doc += f"{analysis['description']}.\n\n"

        doc += "Parameters\n"
        doc += "----------\n"

        for p in signature["params"]:
            doc += f"{p['name']} : {p['type']}\n"
            doc += "    Parameter description.\n"

        doc += "\nReturns\n"
        doc += "-------\n"
        doc += f"{signature['return_type']}\n"
        doc += "    Result of the function.\n"

        doc += '"""'

        return doc
```

```
def generate_docstring(code, style):

    signature = DocGenieAnalyzer.extract_function_signature(code)
    analysis = DocGenieAnalyzer.analyze_function_logic(code)

    if signature is None:
        return "No function detected."

    if style == "Google":
        docstring = DocGenieAnalyzer.generate_google_docstring(signature, analysis)
    else:
        docstring = DocGenieAnalyzer.generate_numpy_docstring(signature, analysis)

    lines = code.split("\n")

    for i, line in enumerate(lines):
        if line.strip().startswith("def"):
            lines.insert(i+1, "    " + docstring)
            break
```

```
    return "\n".join(lines)
```

```
def export_txt(code):

    file_path = tempfile.NamedTemporaryFile(delete=False, suffix=".txt").name

    with open(file_path,"w") as f:
        f.write(code)

    return file_path
```

```
def export_pdf(code):

    file_path = tempfile.NamedTemporaryFile(delete=False, suffix=".pdf").name

    styles = getSampleStyleSheet()
    story = []

    story.append(Paragraph("Doc Genie Generated Documentation", styles["Title"]))
    story.append(Spacer(1,20))

    story.append(Paragraph(f"Generated on {datetime.datetime.now()}", styles["Normal"]))
    story.append(Spacer(1,20))

    for line in code.split("\n"):
        story.append(Paragraph(line.replace(" "," "), styles["Code"]))

    pdf = SimpleDocTemplate(file_path)
    pdf.build(story)

    return file_path
```

```
with gr.Blocks() as demo:

    gr.Markdown("# Doc Genie - Python Docstring Generator")

    with gr.Row():

        code_input = gr.Textbox(
            label="Enter Python Function",
            lines=15,
            value="""def add(a,b):
    return a+b"""
        )

        output = gr.Textbox(
            label="Generated Code",
            lines=15
        )

    style = gr.Radio(["Google","NumPy"], value="Google", label="Docstring Style")

    generate_btn = gr.Button("Generate Docstring")

    generate_btn.click(
        generate_docstring,
        inputs=[code_input,style],
        outputs=output
    )

    txt_btn = gr.Button("Download TXT")
    pdf_btn = gr.Button("Download PDF")

    txt_file = gr.File()
    pdf_file = gr.File()

    txt_btn.click(export_txt, inputs=output, outputs=txt_file)
    pdf_btn.click(export_pdf, inputs=output, outputs=pdf_file)

demo.launch(share=True)
```

Start coding or generate with AI.